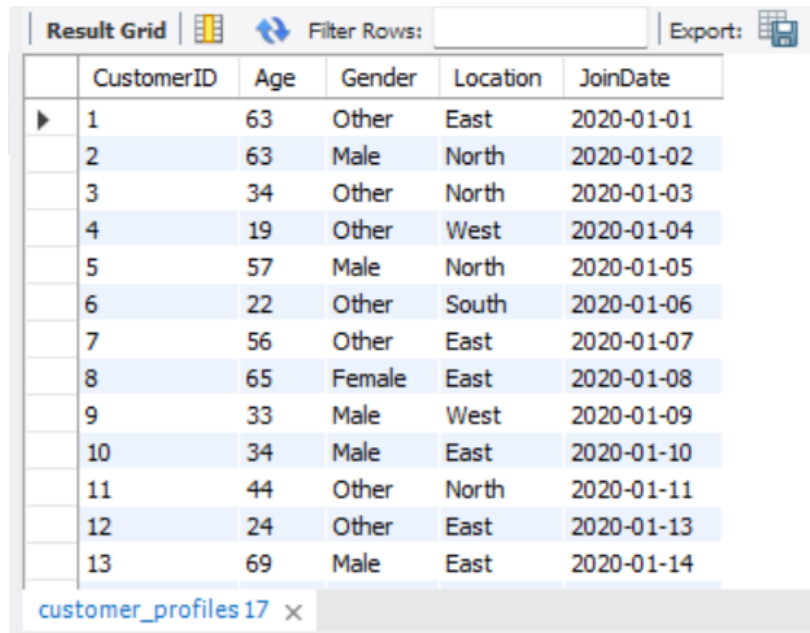


SCREENSHOTS OF SQL QUERY OUTPUTS

1. Select Statement - Used to retrieve specific columns or rows from a table

```
select * from customer_profiles;
```

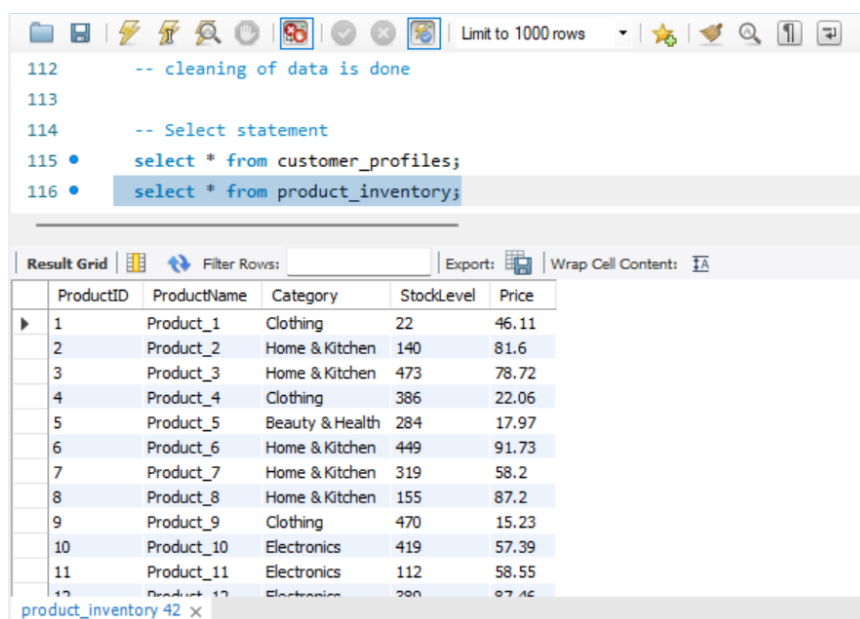


The screenshot shows a SQL query result grid for the 'customer_profiles' table. The grid has a toolbar at the top with 'Result Grid', 'Filter Rows', and 'Export' options. The table contains 13 rows of data with columns: CustomerID, Age, Gender, Location, and JoinDate. The status bar at the bottom indicates 'customer_profiles 17 x'.

	CustomerID	Age	Gender	Location	JoinDate
▶	1	63	Other	East	2020-01-01
	2	63	Male	North	2020-01-02
	3	34	Other	North	2020-01-03
	4	19	Other	West	2020-01-04
	5	57	Male	North	2020-01-05
	6	22	Other	South	2020-01-06
	7	56	Other	East	2020-01-07
	8	65	Female	East	2020-01-08
	9	33	Male	West	2020-01-09
	10	34	Male	East	2020-01-10
	11	44	Other	North	2020-01-11
	12	24	Other	East	2020-01-13
	13	69	Male	East	2020-01-14

customer_profiles 17 x

```
select * from product_inventory;
```



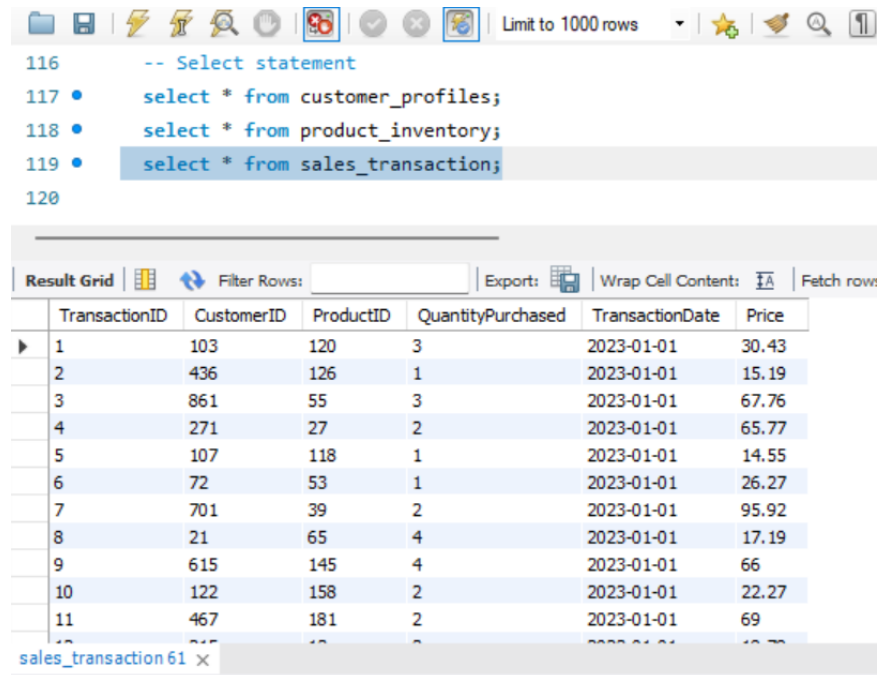
The screenshot shows a SQL query result grid for the 'product_inventory' table. The grid has a toolbar at the top with various icons and a 'Limit to 1000 rows' dropdown. The table contains 12 rows of data with columns: ProductID, ProductName, Category, StockLevel, and Price. The status bar at the bottom indicates 'product_inventory 42 x'.

```
112 -- cleaning of data is done
113
114 -- Select statement
115 • select * from customer_profiles;
116 • select * from product_inventory;
```

	ProductID	ProductName	Category	StockLevel	Price
▶	1	Product_1	Clothing	22	46.11
	2	Product_2	Home & Kitchen	140	81.6
	3	Product_3	Home & Kitchen	473	78.72
	4	Product_4	Clothing	386	22.06
	5	Product_5	Beauty & Health	284	17.97
	6	Product_6	Home & Kitchen	449	91.73
	7	Product_7	Home & Kitchen	319	58.2
	8	Product_8	Home & Kitchen	155	87.2
	9	Product_9	Clothing	470	15.23
	10	Product_10	Electronics	419	57.39
	11	Product_11	Electronics	112	58.55
	12	Product_12	Electronics	380	87.46

product_inventory 42 x

select * from sales_transaction;



The screenshot shows a database query editor with a toolbar at the top. The SQL statement is as follows:

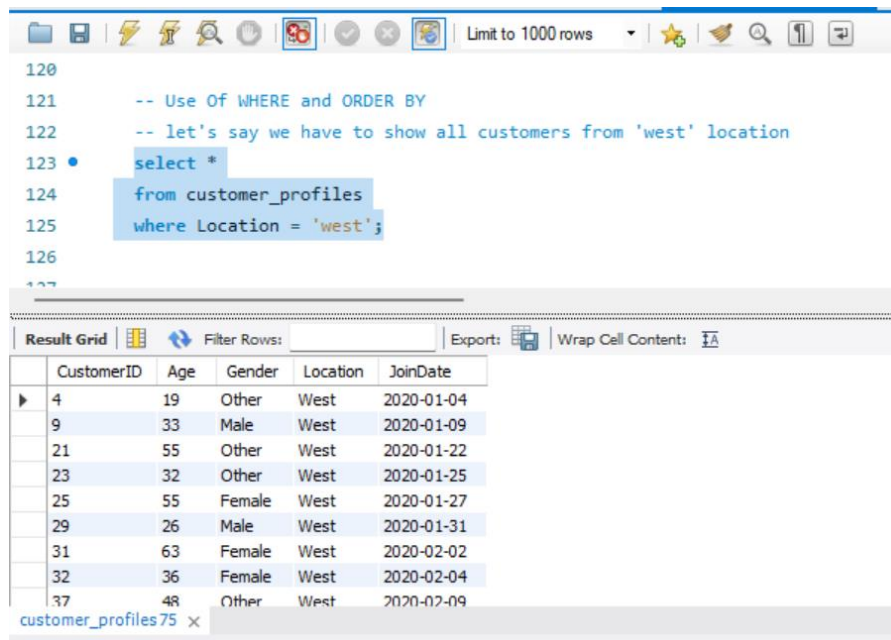
```
116 -- Select statement
117 • select * from customer_profiles;
118 • select * from product_inventory;
119 • select * from sales_transaction;
120
```

The result grid below the statement displays the following data:

	TransactionID	CustomerID	ProductID	QuantityPurchased	TransactionDate	Price
▶	1	103	120	3	2023-01-01	30.43
	2	436	126	1	2023-01-01	15.19
	3	861	55	3	2023-01-01	67.76
	4	271	27	2	2023-01-01	65.77
	5	107	118	1	2023-01-01	14.55
	6	72	53	1	2023-01-01	26.27
	7	701	39	2	2023-01-01	95.92
	8	21	65	4	2023-01-01	17.19
	9	615	145	4	2023-01-01	66
	10	122	158	2	2023-01-01	22.27
	11	467	181	2	2023-01-01	69

2. Where - Filters rows based on a given condition

-- Q. we have to show all customers from 'west' location



The screenshot shows a database query editor with a toolbar at the top. The SQL statement is as follows:

```
120
121 -- Use Of WHERE and ORDER BY
122 -- let's say we have to show all customers from 'west' location
123 • select *
124   from customer_profiles
125   where Location = 'west';
126
127
```

The result grid below the statement displays the following data:

	CustomerID	Age	Gender	Location	JoinDate
▶	4	19	Other	West	2020-01-04
	9	33	Male	West	2020-01-09
	21	55	Other	West	2020-01-22
	23	32	Other	West	2020-01-25
	25	55	Female	West	2020-01-27
	29	26	Male	West	2020-01-31
	31	63	Female	West	2020-02-02
	32	36	Female	West	2020-02-04
	37	48	Other	West	2020-02-09

3. ORDER BY - Sorts the result set in ascending or descending order.

-- Q. find the list of products priced above 50, sorted from highest to lowest price.

Limit to 1000 rows

```

127 -- Use of ORDER BY
128 -- let's say we have to find the list of products priced above 50,
129 -- sorted from highest to lowest price.
130
131 • Select ProductID, ProductName, Price
132 From product_inventory
133 Where Price > 50
134 Order by Price DESC;
135
...

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	ProductID	ProductName	Price
▶	85	Product_85	99.81
	36	Product_36	99.46
	106	Product_106	99.41
	18	Product_18	99.35
	96	Product_96	99.06
	124	Product_124	97.86
	75	Product_75	96.74
	19	Product_19	96.33
	39	Product_39	95.92
	180	Product_180	94.76

product_inventory 78 x

4. GROUP BY with Aggregate Functions - Groups rows with the same values for aggregate calculations

- SUM

-- Q. Find the total quantity sold for each product.

Limit to 1000 rows

```

135
136 -- GROUP BY with Aggregate Functions
137 -- Find the total quantity sold for each product
138 -- I use group by with sum(Aggregate function)
139
140 • Select ProductID, SUM(QuantityPurchased) AS TotalQuantitySold
141 From sales_transaction
142 Group by ProductID;
143
...

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	ProductID	TotalQuantitySold
▶	1	40
	2	64
	3	48
	4	58
	5	47
	6	51
	7	61
	8	65
	9	60
	10	79

Result 79 x

- AVG

-- Q. Find the average age of customers per location.

The screenshot shows a database management tool interface. On the left, the 'SCHEMAS' panel displays a tree view with 'phpmyadmin' and 'retails' (containing 'customer_profiles', 'product_inventory', and 'sales_transaction'). Below this, the 'Table: customer_profiles' is detailed with columns: CustomerID (int(11)), Age (int(11)), Gender (text), Location (text), and JoinDate (date). The main SQL editor contains the following query:

```
143
144 -- Find the average age of customers per location.
145 • Select Location, AVG(Age) AS Avg_age
146 From customer_profiles
147 Group by Location;
148
```

Below the query, the 'Result Grid' shows the following data:

Location	Avg_age
East	44.2800
North	45.2298
South	42.6074
West	43.3556

- MIN & MAX

-- Q. For each product, find the minimum and maximum quantity purchased in a single transaction.

The screenshot shows a database management tool interface. The SQL editor contains the following query:

```
155 -- For each product, find the minimum and maximum
156 -- quantity purchased in a single transaction.
157
158 • Select
159     ProductID,
160     MIN(QuantityPurchased) AS MinQtyPurchased,
161     MAX(QuantityPurchased) AS MaxQtyPurchased
162 From sales_transaction
163 Group by ProductID;
```

Below the query, the 'Result Grid' shows the following data:

ProductID	MinQtyPurchased	MaxQtyPurchased
1	1	4
2	1	4
3	1	4
4	1	4
5	1	4
6	1	4
7	1	4
8	1	4
9	1	4
10	1	4

5. JOINS - JOIN is a clause that combines rows from two or more tables based on a related column between them.

- **LEFT JOIN** - Returns all rows from the left table & matching rows from the right table (NULL if no match).

-- Q. Show all products and their total quantity sold

The screenshot shows the phpMyAdmin interface. On the left, the 'SCHEMAS' panel is open, showing the 'retails' database with tables 'customer_profiles', 'product_inventory', and 'sales_transaction'. The 'customer_profiles' table structure is displayed below it. The main SQL editor shows a query using a LEFT JOIN between 'product_inventory' and 'sales_transaction' to calculate the total quantity sold for each product. The 'Result Grid' shows the output of the query.

```
165 -- Use of JOINS
166 -- LEFT JOIN
167 -- Show all products and their total quantity sold
168 -- Using LEFT JOIN
169
170 • Select p.ProductName, SUM(s.QuantityPurchased) AS TotalQuantitySold
171 From product_inventory p
172 Left Join sales_transaction s ON p.ProductID = s.ProductID
173 Group by p.ProductName;
```

ProductName	TotalQuantitySold
Product_120	64
Product_121	46
Product_122	71
Product_123	58
Product_124	39
Product_125	79
Product_126	71
Product_127	68
Product_128	64
Product_129	71

- **RIGHT JOIN** - Returns all rows from the right table & matching rows from the left table (NULL if no match).

-- Q. Find all customers who have not purchased anything

The screenshot shows the phpMyAdmin interface. On the left, the 'SCHEMAS' panel is open, showing the 'retails' database with tables 'customer_profiles', 'product_inventory', and 'sales_transaction'. The 'customer_profiles' table structure is displayed below it. The main SQL editor shows a query using a RIGHT JOIN between 'customer_profiles' and 'sales_transaction' to find customers who have not purchased anything. The 'Result Grid' shows the output of the query.

```
173 Group by p.ProductName;
174
175 -- RIGHT JOIN
176 -- Find all customers who have not purchased anything
177
178 • Select c.CustomerID, c.Location
179 From sales_transaction s
180 Right Join customer_profiles c ON s.CustomerID = c.CustomerID
181 Where s.TransactionID IS NULL;
```

CustomerID	Location
52	East
71	South
197	North
299	West
433	North
600	West
671	South
694	East
756	North
765	South

- **INNER JOIN** - Returns rows where there is a match in both tables.

-- Q. Find total quantity sold per location

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'SCHEMAS' pane displays the 'retails' database structure, including tables like 'customer_profiles', 'product_inventory', and 'sales_transaction'. The 'customer_profiles' table is selected, showing its columns: CustomerID (int(11)), Age (int(11)), Gender (text), Location (text), and JoinDate (date). The main pane displays a SQL query:

```
182
183 -- INNER JOIN
184 -- Find total quantity sold per location
185
186 • Select c.Location, SUM(s.QuantityPurchased) AS TotalQuantitySold
187 From sales_transaction s
188 Inner Join customer_profiles c
189 ON s.CustomerID = c.CustomerID
190 Group by c.Location
191 Order by TotalQuantitySold DESC;
192
193
```

Below the query, the 'Result Grid' shows the following data:

Location	TotalQuantitySold
West	3589
North	3037
South	2905
East	2791

- **SELF JOIN** - Joins a table to itself to compare rows within the same table.

-- Q. Find customers from the same location

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'SCHEMAS' pane displays the 'retails' database structure, including tables like 'customer_profiles', 'product_inventory', and 'sales_transaction'. The 'customer_profiles' table is selected, showing its columns: CustomerID (int(11)), Age (int(11)), Gender (text), Location (text), and JoinDate (date). The main pane displays a SQL query:

```
192
193 -- SELF JOIN
194 -- Find customers from the same location
195
196 • Select c1.CustomerID AS Customer1, c2.CustomerID AS Customer2, c1.Location
197 From customer_profiles c1
198 Join customer_profiles c2
199 ON c1.Location = c2.Location
200 Where c1.CustomerID <> c2.CustomerID;
201
```

Below the query, the 'Result Grid' shows the following data:

Customer1	Customer2	Location
7	1	East
8	1	East
10	1	East
12	1	East
13	1	East
16	1	East
27	1	East
28	1	East
38	1	East
47	1	East

6. SUBQUERIES - A query nested inside another query to provide intermediate results

-- Q. Get all customers who spent more than the average spending of all customers

203 -- USE OF SUBQUERIES
 204 -- Get all customers who spent more than the average spending of all customers
 205
 206 • Select CustomerID, ROUND(SUM(Price * QuantityPurchased), 2) AS TotalSpent,
 207 ROUND((Select AVG(CustomerTotal)
 208 From (Select SUM(Price * QuantityPurchased) AS CustomerTotal
 209 From sales_transaction
 210 Group by CustomerID) AS avg_spending), 2) AS AverageSpending
 211 From sales_transaction
 212 Group by CustomerID
 213 Having TotalSpent > (
 214 Select AVG(CustomerTotal)
 215 From (

Result Grid

	CustomerID	TotalSpent	AverageSpending
▶	1	1188.04	709.81
	2	895.57	709.81
	7	1383.86	709.81
	8	1297.25	709.81
	12	1426.31	709.81
	13	927.24	709.81
	15	748.95	709.81
	16	825.81	709.81

Result 3 x

-- Q. Find the top 3 most expensive products using a subquery in Where.

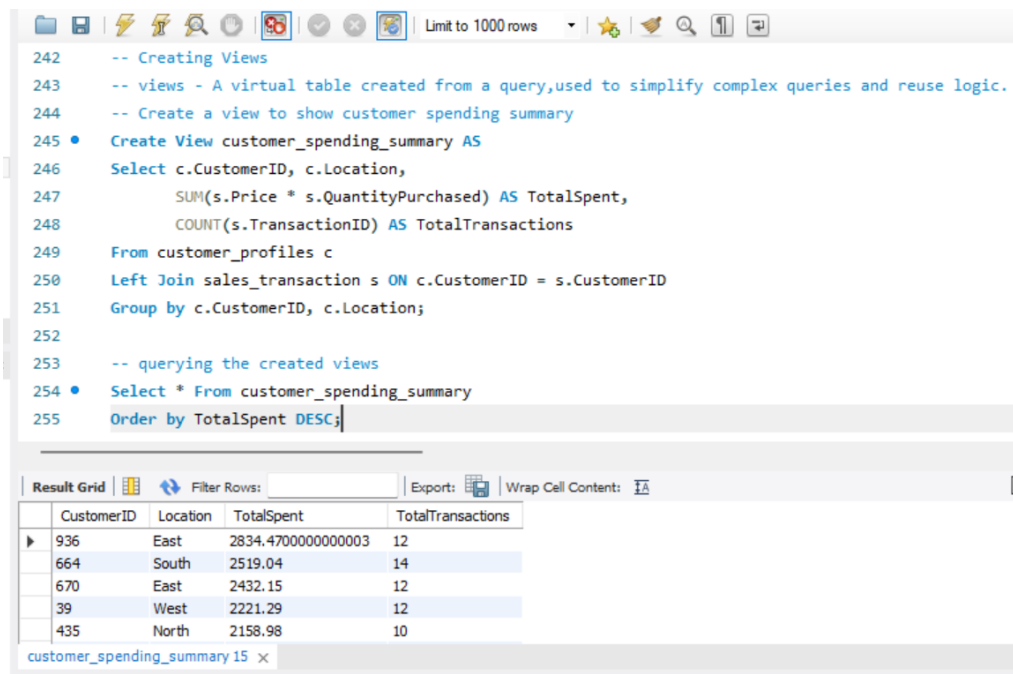
223 • Select ProductName, Price
 224 From product_inventory
 225 Where Price IN (
 226 Select Price
 227 From (
 228 Select Price
 229 From product_inventory
 230 Order by Price DESC
 231 Limit 3
 232) AS top_prices
 233)
 234 Order by Price DESC;
 235

Result Grid

	ProductName	Price
▶	Product_85	99.81
	Product_36	99.46
	Product_106	99.41

7. VIEWS - A virtual table created from a query, used to simplify complex queries and reuse logic.

-- Q. Create a view to show customer spending summary & their result



The screenshot shows a SQL IDE window with a toolbar at the top. The main text area contains SQL code for creating and querying a view. The code is as follows:

```
242 -- Creating Views
243 -- views - A virtual table created from a query,used to simplify complex queries and reuse logic.
244 -- Create a view to show customer spending summary
245 • Create View customer_spending_summary AS
246   Select c.CustomerID, c.Location,
247         SUM(s.Price * s.QuantityPurchased) AS TotalSpent,
248         COUNT(s.TransactionID) AS TotalTransactions
249   From customer_profiles c
250  Left Join sales_transaction s ON c.CustomerID = s.CustomerID
251  Group by c.CustomerID, c.Location;
252
253 -- querying the created views
254 • Select * From customer_spending_summary
255   Order by TotalSpent DESC;
```

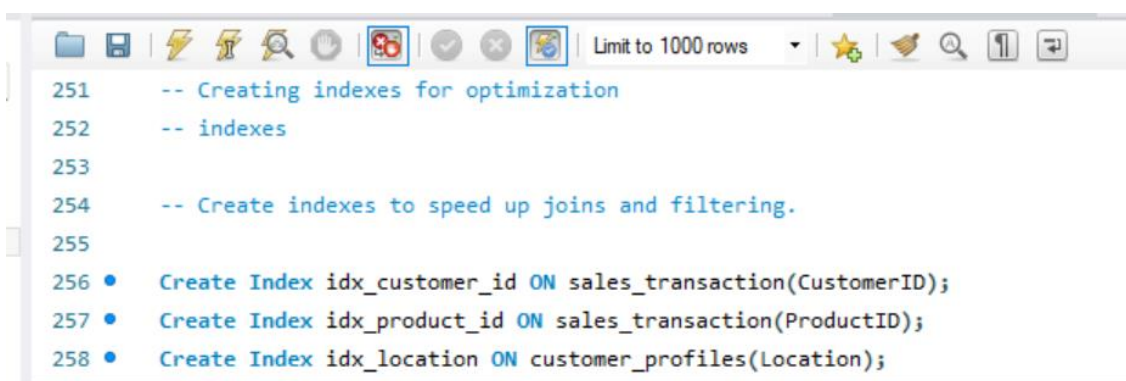
Below the code editor, the 'Result Grid' is displayed, showing the output of the query. It includes a table with the following data:

CustomerID	Location	TotalSpent	TotalTransactions
936	East	2834.4700000000003	12
664	South	2519.04	14
670	East	2432.15	12
39	West	2221.29	12
435	North	2158.98	10

The bottom of the window shows a tab labeled 'customer_spending_summary 15 x'.

8. INDEXES - Indexing in SQL involves creating a data structure to improve the speed of data retrieval operations from a database table.

-- Create indexes to speed up joins and filtering.

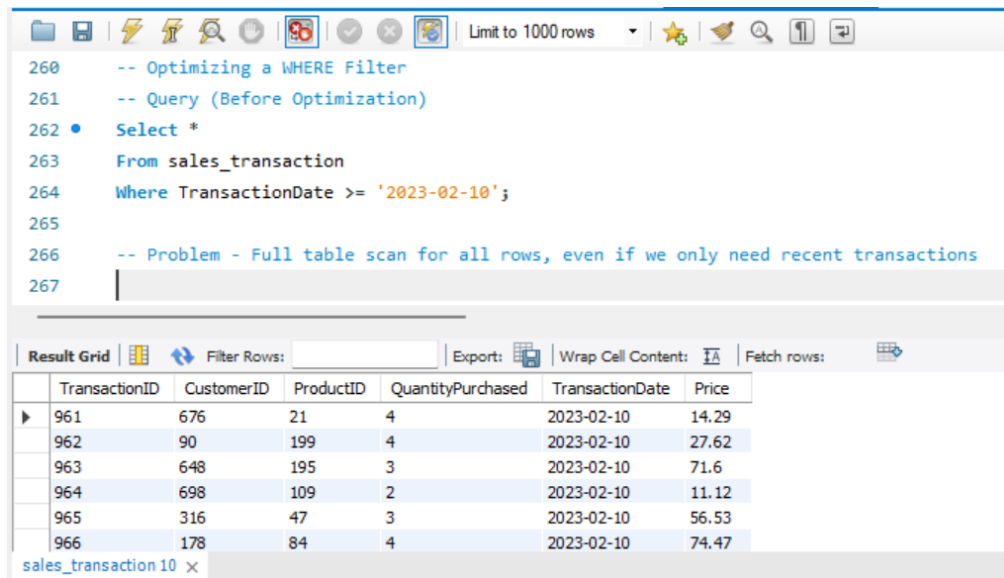


The screenshot shows a SQL IDE window with a toolbar at the top. The main text area contains SQL code for creating three indexes. The code is as follows:

```
251 -- Creating indexes for optimization
252 -- indexes
253
254 -- Create indexes to speed up joins and filtering.
255
256 • Create Index idx_customer_id ON sales_transaction(CustomerID);
257 • Create Index idx_product_id ON sales_transaction(ProductID);
258 • Create Index idx_location ON customer_profiles(Location);
```


-- Optimizing a WHERE Filter Using Indexing

-- Query (Before Optimization)



The screenshot shows a SQL query editor with a toolbar at the top. The query text is as follows:

```
260 -- Optimizing a WHERE Filter
261 -- Query (Before Optimization)
262 • Select *
263 From sales_transaction
264 Where TransactionDate >= '2023-02-10';
265
266 -- Problem - Full table scan for all rows, even if we only need recent transactions
267
```

Below the query editor is a 'Result Grid' section. It includes a 'Filter Rows' input field, an 'Export' button, a 'Wrap Cell Content' checkbox, and a 'Fetch rows' button. The grid displays the following data:

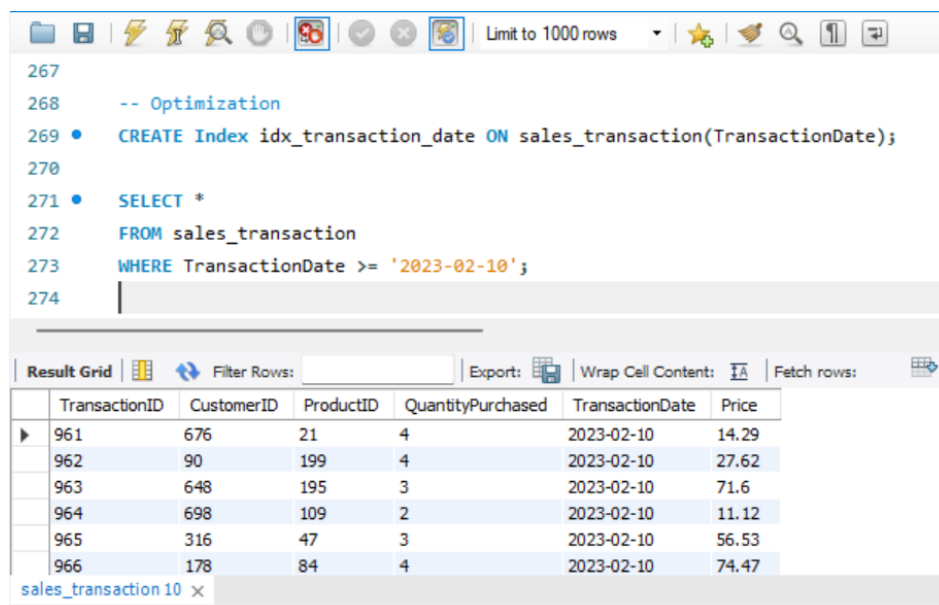
TransactionID	CustomerID	ProductID	QuantityPurchased	TransactionDate	Price
961	676	21	4	2023-02-10	14.29
962	90	199	4	2023-02-10	27.62
963	648	195	3	2023-02-10	71.6
964	698	109	2	2023-02-10	11.12
965	316	47	3	2023-02-10	56.53
966	178	84	4	2023-02-10	74.47

The table is titled 'sales_transaction 10'.

-- **Problem** – Full table scan for all rows, even if we only need recent transactions date

-- The query take 0.015 sec time duration to process & 0.016 sec time in Fetch the result

-- After Optimization



The screenshot shows the same SQL query editor with the following optimized query:

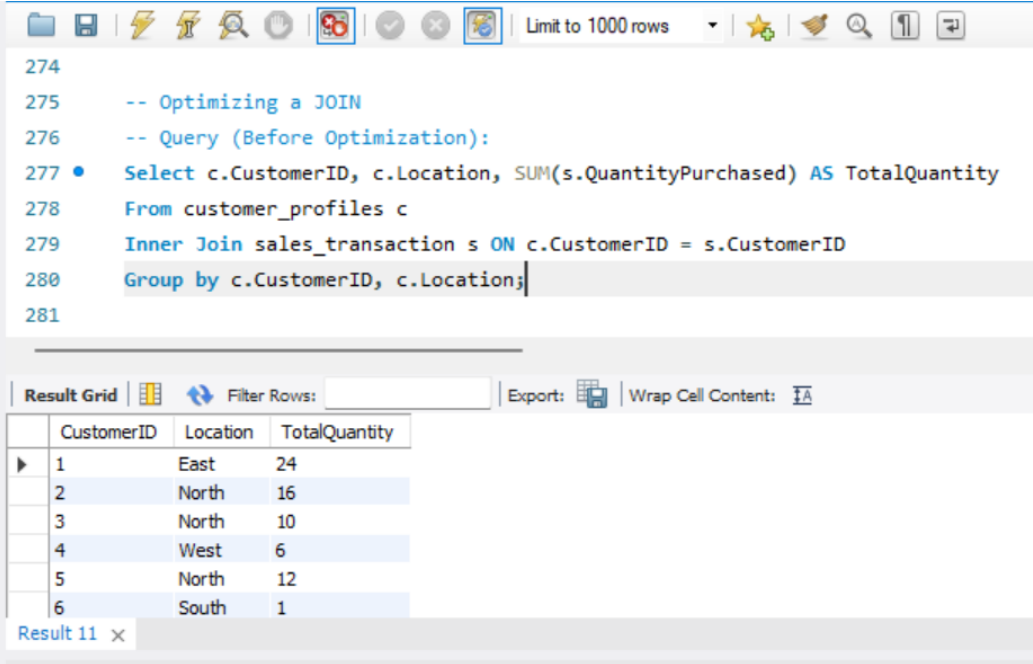
```
267
268 -- Optimization
269 • CREATE Index idx_transaction_date ON sales_transaction(TransactionDate);
270
271 • SELECT *
272 FROM sales_transaction
273 WHERE TransactionDate >= '2023-02-10';
274
```

The 'Result Grid' section is identical to the previous screenshot, displaying the same data table.

-- Now the query take 0.000 sec time duration to process & 0.016 sec time in Fetch the result

-- Optimizing a JOIN Using Indexing

-- Query (Before Optimization)



The screenshot shows a SQL IDE window with a toolbar at the top. The query editor contains the following SQL code:

```
274
275 -- Optimizing a JOIN
276 -- Query (Before Optimization):
277 • Select c.CustomerID, c.Location, SUM(s.QuantityPurchased) AS TotalQuantity
278 From customer_profiles c
279 Inner Join sales_transaction s ON c.CustomerID = s.CustomerID
280 Group by c.CustomerID, c.Location;
281
```

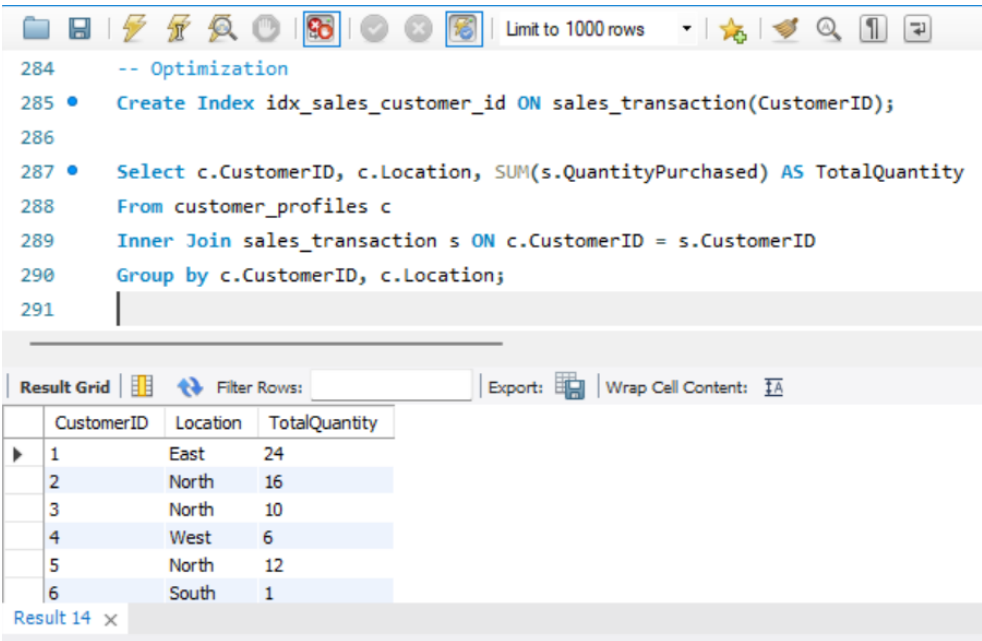
Below the query editor is a 'Result Grid' section. It includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The grid displays the following data:

	CustomerID	Location	TotalQuantity
▶	1	East	24
	2	North	16
	3	North	10
	4	West	6
	5	North	12
	6	South	1

The status bar at the bottom indicates 'Result 11'.

Problem - Joins are slow if sales_transaction is big.

-- After Optimization



The screenshot shows the same SQL IDE window after optimization. The query editor contains the following SQL code:

```
284 -- Optimization
285 • Create Index idx_sales_customer_id ON sales_transaction(CustomerID);
286
287 • Select c.CustomerID, c.Location, SUM(s.QuantityPurchased) AS TotalQuantity
288 From customer_profiles c
289 Inner Join sales_transaction s ON c.CustomerID = s.CustomerID
290 Group by c.CustomerID, c.Location;
291
```

The 'Result Grid' section is identical to the previous screenshot, displaying the same data:

	CustomerID	Location	TotalQuantity
▶	1	East	24
	2	North	16
	3	North	10
	4	West	6
	5	North	12
	6	South	1

The status bar at the bottom indicates 'Result 14'.